

JAWAHARLAL COLLEGE
OF ENGINEERING AND TECHNOLOGY
Jawahar Gardens, Lakkidi, Mangalam, Palakkad 679-301
(An Autonomous Institution Affiliated to APJ-Abdul Kalam Technological University)
Approved by AICTE | Accredited by NAAC "A+"
NBA Accredited Programmes | ISO 9001:2015 Certified Institution



REMOTE NETWORK ACCESS USING TWIN GATE AND RASPBERRY PI

A Seminar Report submitted in partial fulfilment of the requirement for the

award of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING(CYBERSECURITY)

under

APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY

by

AKSHAY N- JCE23CC008

Guided by

Ms. FARBEENA N.

Assistant Professor – CSE

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING(CYBERSECURITY)

JAWAHARLAL COLLEGE OF ENGINEERING & TECHNOLOGY

LAKKIDI, MANGALAM P.O, PALAKKAD, KERALA – 679301

OCTOBER 2026

JAWAHARLAL COLLEGE
OF ENGINEERING AND TECHNOLOGY
Jawahar Gardens, Lakkidi, Mangalam, Palakkad 679-301
(An Autonomous Institution Affiliated to APJ-Abdul Kalam Technological University)
Approved by AICTE | Accredited by NAAC "A+"
NBA Accredited Programmes | ISO 9001:2015 Certified Institution

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (CYBERSECURITY)

CERTIFICATE

This is to certify that this is the Bonafide report of the seminar titled '**REMOTE NETWORK ACCESS USING TWIN GATE AND RASPBERRY PI**' done by **AKSHAY N** of **Reg. No. JCE23CC008** in partial fulfilment of the requirements for the award of **Bachelor of Technology in Computer Science and Engineering** under **APJ – Abdul Kalam Technological University**.

SUPERVISOR	COORDINATOR	HEAD OF THE DEPARTMENT
Ms. FARBEENA N	**Mr. MANOJ M.**	**Mr. MANOJ M**
Assistant Professor	Assistant Professor	Assistant Professor

INSTITUTE VISION

Emerge as a Centre of excellence for professional education to produce high quality engineers and entrepreneurs for the development of the region and the Nation.

INSTITUTE MISSION

- To become an ultimate destination for acquiring latest and advanced knowledge in the multidisciplinary domains.
- To provide high quality education in engineering and technology through innovative teaching-learning practices, research and consultancy, embedded with professional ethics.
- To promote intellectual curiosity and thirst for acquiring knowledge through outcomebased education.
- To have partnership with industry and reputed institutions to enhance the employability skills of the students and pedagogical pursuits.
- To leverage technologies to solve the real-life societal problems through community services.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (CYBERSECURITY)

VISION

To produce competent professionals with research and innovative skills, by providing them with most conducive environment for quality academic and research oriented undergraduate and postgraduate education along with moral values committed to build a vibrant nation.

MISSION

- Provide a learning environment to develop creativity and problem-solving skills in a professional manner.
- Expose to latest technologies and tools used in the field of computer science.
- Provide a platform to explore the industries to understand the work culture and expectation of an organization.
- Enhance Industry Institute Interaction program to develop the entrepreneurship skills.
- Develop research interest among students which will impart a better life for the society and the nation.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (CYBERSECURITY)

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

The Graduates in Computer Science and Engineering will be able to:

PEO1: Provide high quality knowledge in computer science and engineering required for a computer professional to identify and solve problems in various application domains.

PEO2: Persist with the ability in innovative ideas in computer support systems and transmit the knowledge and skills for research and advanced learning.

PEO3: Manifest the motivational capabilities and turn on social and economic commitment to community services.

PROGRAM SPECIFIC OUTCOMES (PSOs)

PSO1: Use fundamental knowledge of mathematics to solve problems using suitable analysis methods, data structure and algorithms.

PSO2: Interpret the basic concepts and methods of computer systems and technical specifications to provide accurate solutions.

PSO3: Apply theoretical and practical proficiency with a wide area of programming knowledge, design new ideas

The following are the Program Outcomes of Engineering Students:

PO1	**Engineering knowledge**: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
PO2	**Problem analysis**: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
PO3	**Design/development of solutions**: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
PO4	**Conduct investigations of complex problems**: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
PO5	**Modern tool usage**: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
PO6	**The engineer and society**: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
PO7	**Environment and sustainability**: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
PO8	**Ethics**: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9	**Individual and team work**: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
PO10	**Communication**: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
PO11	**Project management and finance**: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments
PO12	**Life-long learning**: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

ACKNOWLEDGEMENT

I thank Almighty God for giving me the blessings to complete my seminar successfully. I sincerely appreciate the inspiration support and guidance of all those people who have been instrumental in making this seminar a success.

I would like to sincerely thank **Adv. Dr. P. Krishnadas**, Managing Trustee of Nehru college Educational and Charitable Trust for making the resources available at right time and providing valuable insights leading to the successful completion of seminar.

I express my sincere thanks to **Dr. N. Gunasekaran**, The Principal of our college for supporting all the way long.

I express my special gratitude to **Mr. Manoj M**, Head of the Department of Computer Science and Engineering for providing me constant guidance and encouragement throughout the seminar.

I express my sincere gratitude to the Seminar Coordinator, **Mr. Manoj M**, Assistant Professor, Department of Computer Science and Engineering for his inspiration and timely suggestions.

I also express sincere gratitude to my Seminar Guide **Ms. Farbeena N**, Assistant Professor, Department of Computer Science and Engineering for his guidance and support.

AKSHAY N

TABLE OF CONTENTS

CHAPTER NO.		TITLE	PAGE NO
		ABSTRACT	i
		LIST OF TABLES	ii
		LIST OF FIGURES	iii
		LIST OF ABBREVIATIONS	iv
1		**INTRODUCTION**	
	1.1	BACKGROUND	1
	1.2	MOTIVATION AND SIGNIFICANCE	2
	1.3	OBJECTIVES	2
	1.4	SCOPE OF THE STUDY	3
	1.5	IMPORTANCE OF REMOTE NETWORK ACCESS USING TWIN GATE AND RASPBERRY PI	3
2		**SYSTEM ANALYSIS**	
	2.1	LITERATURE SURVEY REVIEW	4
	2.2	EXISTING SYSTEMS	5
	2.2.1	TRADITIONAL SSL/IPSEC VPN SOLUTIONS	5
	2.2.2	PROPRIETARY ENTERPRISE ZTNA APPLIANCES	5
	2.2.3	INTERMEDIATE BASTION HOSTS AND JUMP SERVERS	5
	2.3	PROPOSED SYSTEM	5
	2.4	SYSTEM ARCHITECTURE OVERVIEW	6
	2.5	MODULE DESCRIPTION	8
	2.5.1	HARDWARE CONNECTOR & NETWORK INTEGRATION MODULE	8

	2.5.2	TWINGATE ZERO-TRUST ACCESS & IDENTITY CONTROL MODULE	8
	2.5.3	AUTOMATED RESOURCE DISCOVERY & REGISTRATION MODULE	8
	2.5.4	ADMIN & SECURITY SERVICES MANAGEMENT MODULE	8
	2.6	SYSTEM ADVANTAGES	9
3		**SYSTEM REQUIREMENTS**	
	3.1	INTRODUCTION	10
	3.2	HARDWARE REQUIREMENTS	10
	3.3	SOFTWARE REQUIREMENTS	11
	3.4	FUNCTIONAL REQUIREMENTS	13
	3.5	NON-FUNCTIONAL REQUIREMENTS	14
4		**SYSTEM DESIGN AND IMPLEMENTATION**	
	4.1	OVERVIEW	15
	4.2	SYSTEM ARCHITECTURE	15
	4.3	WORKFLOW DESCRIPTION	16
	4.4	MODULES AND THEIR INTERACTIONS	17
	4.4.1	HARDWARE & OS SETUP MODULE	18
	4.4.2	TWINGATE & ZTNA INTEGRATION MODULE	18
	4.4.3	NETWORK DISCOVERY & AUTOMATION MODULE	18
	4.4.4	ACCESS CONTROL & POLICY MODULE	19
	4.4.5	CLIENT & MONITORING INTEGRATION MODULE	19
	4.5	IMPLEMENTATION DETAILS	20
	4.6	CHALLENGES AND MITIGATION STRATEGIES	21
5		**ETHICAL CONSIDERATIONS AND LIMITATIONS**	

	5.1	INTRODUCTION	22
	5.2	ETHICAL CONSIDERATIONS	22
	5.3	LIMITATIONS OF THE SYSTEM	23
	5.4	MITIGATION STRATEGIES	24
	5.5	SUMMARY	25
6		**CONCLUSION AND FUTURE SCOPE**	
	6.1	CONCLUSION	26
	6.2	FUTURE SCOPE	26
		REFERENCES	28
		APPENDIX	29

ABSTRACT

The proposed system focuses on developing a secure, efficient, and cost-effective solution for remote network access by using a Raspberry Pi as a connector for the Twingate Zero Trust Network Access (ZTNA) platform. Traditional remote access methods, such as Virtual Private Networks (VPNs), often require firewall modifications, port forwarding, public IP addresses, and complex network configurations. This project addresses these limitations by implementing a zero-trust approach based on identity verification and least-privilege access, ensuring users can only access authorized resources. The Raspberry Pi functions as a lightweight and affordable network connector that securely links remote devices and services to the cloud-managed Twingate platform, which simplifies deployment, centralized management, and authentication without exposing internal network services directly to the internet. Security is reinforced through granular access policies, device verification, and activity logging. To improve scalability, the system integrates the Twingate API with a Python-based automation script to dynamically discover and register available network resources. Additional features like DNS aliasing, temporary access permissions, and service accounts further support secure remote administration, network monitoring, and authorized security assessments.

LIST OF TABLES

TABLE NO	TITLE	PAGE NO.
1	HARDWARE REQUIREMENTS	13
2	SOFTWARE REQUIREMENTS	14
3	CHALLENGES AND MITIGATION STRATEGIES	26
4	MITIGATION STRATEGIES	30

LIST OF FIGURES

FIGURE NO	TITLE	PAGE NO
1	SYSTEM ARCHITECTURE	9
2	OVERALL SYSTEM WORKFLOW	22
3	TWINGATE INTEGRATION MODULE	23
4	NETWORK POLICY MODULE	23
5	AUTHARIZATION MODULE	24
6	MONITORING AND LOGGING MODULE	24

LIST OF ABBREVIATIONS

ABBREVIATION	FULL FORM
ZTNA	ZERO TRUST NETWORK ACCESS
VPN	VIRTUAL PRIVATE NETWORK
IP	INTERNET PROTOCOL
API	APPLICATION PROGRAMMING INTERFACE
DNS	DOMAIN NAME SYSTEM
SSH	SECURE SHELL
RDP	REMOTE DESKTOP PROTOCOL
LAN	LOCAL AREA NETWORK
WAN	WIDE AREA NETWORK
NAT	NETWORK ADDRESS TRANSLATION
IAM	IDENTITY AND ACCESS MANAGEMENT
MFA	MULTI-FACTOR AUTHENTICATION
HTTP	HYPERTEXT TRANSFER PROTOCOL
HTTPS	HYPERTEXT TRANSFER PROTOCOL SECURE
TCP	TRANSMISSION CONTROL PROTOCOL
UDP	USER DATAGRAM PROTOCOL

CHAPTER 1
INTRODUCTION

1.1 BACKGROUND

The shift toward distributed networks and remote administration has exposed significant limitations in traditional perimeter-based security models. Historically, organizations have relied on Virtual Private Networks (VPNs) to facilitate remote access. However, VPNs typically grant broad, network-level access once a user is authenticated, inherently violating the principle of least privilege and increasing the potential impact of compromised credentials. Additionally, maintaining traditional remote access infrastructure requires complex network configurations, including firewall modifications, port forwarding, and the allocation of public IP addresses. These requirements not only increase administrative overhead but also expose internal network services directly to the public internet, creating a broader external attack surface.

To address these vulnerabilities, modern cybersecurity engineering is transitioning toward Zero Trust Network Access (ZTNA) architectures. The zero-trust model operates on the foundational principle that no user or device is inherently trusted, regardless of their location relative to the network perimeter. Access is instead granted dynamically based on strict identity verification, device authorization, and granular, resource-specific policies. This ensures that authenticated users can interact strictly with the specific applications or data they are authorized to use, significantly mitigating the risk of lateral movement within the network.

Despite the security advantages of ZTNA, enterprise-grade deployments often demand expensive proprietary hardware appliances or complex cloud dependencies. There is a distinct need for a localized, cost-effective edge solution capable of bridging on-premises environments with cloud-managed access controls. Leveraging small-form-factor edge devices, such as the Raspberry Pi, provides an opportunity to deploy secure network connectors that minimize physical footprint, power consumption, and financial cost.

This project bridges that gap by utilizing a Raspberry Pi as a lightweight, affordable network connector for the Twingate ZTNA platform.This hardware-software integration creates a secure outbound tunnel to the cloud-managed architecture, simplifying deployment and centralized management without exposing internal ports. By combining this localized hardware approach with Python-based API automation for dynamic resource discovery, the project establishes a scalable, transparent, and easily auditable framework for secure remote access.

1.2 MOTIVATION AND SIGNIFICANCE

The primary motivation for this project stems from the operational and security limitations inherent in traditional remote access methods, such as Virtual Private Networks (VPNs). Standard VPN architectures typically require complex network configurations, including firewall modifications, port forwarding, and the allocation of public IP addresses. These requirements not only increase administrative overhead but also inadvertently expand the attack surface by exposing internal network services directly to the internet. Furthermore, traditional VPNs generally grant broad network-level access upon authentication, which violates the cybersecurity principle of least privilege. There is a growing need for a localized, cost-effective solution that can enforce strict access controls without heavy hardware investments. Utilizing a Raspberry Pi addresses this gap by providing a lightweight and affordable hardware connector to bridge physical networks with cloud-managed security platforms. Additionally, the desire to reduce human error and manual administration drove the need to incorporate API-driven automation for network resource discovery.

This project is significant because it democratizes enterprise-grade Zero Trust Network Access (ZTNA) by combining affordable hardware with a modern cloud-managed architecture. By integrating the Raspberry Pi with the Twingate platform, the proposed system completely shifts the defense model from perimeter-based security to identity-driven access control

1.3 OBJECTIVES

This seminar report aims to:

1. To develop a secure remote network access solution by configuring an affordable Raspberry Pi as a lightweight connector for the Twingate Zero Trust Network Access (ZTNA) platform.
2. To bypass the limitations of traditional Virtual Private Networks (VPNs) by removing the need for firewall modifications, port forwarding, public IP addresses, and complex network configurations.
3. To apply strict identity and device verification, ensuring that users are granted access solely to the specific resources they are authorized to use, without exposing internal network services directly to the public internet.
4. To integrate the Twingate API with a Python-based automation script capable of dynamically discovering and securely registering available network resources, thereby minimizing manual administrative configuration.
5. To facilitate secure network monitoring, maintenance, and authorized security assessments by implementing features such as DNS aliasing, temporary access permissions, service accounts, and detailed activity logging.

1.4 SCOPE OF THE STUDY

The scope of this study encompasses the design, implementation, and evaluation of a Zero Trust Network Access (ZTNA) architecture utilizing a Raspberry Pi and the Twingate platform. The project focuses on replacing traditional perimeter-based remote access methods, such as VPNs, with an identity-driven, least-privilege security model:

- Hardware and Network Integration: Configuring an affordable, lightweight Raspberry Pi to function as a Twingate network connector. This includes establishing secure links between remote devices and internal services

through a cloud-managed architecture without exposing internal ports to the internet, modifying firewalls, or requiring public IP addresses

- Access Control Implementation: Enforcing a zero-trust security model by implementing strict identity verification, device verification, and granular access policies to ensure users can only interact with authorized resources.
- API Integration and Automation: Developing a Python-based automation script integrated with the Twingate API. The scope covers the programming logic required to dynamically discover available network resources and automatically register them, minimizing manual administrative overhead and improving system scalability.

1.5 IMPORTANCE OF REMOTE NETWORK ACCESS USING TWIN GATE AND RASPBERRY PI

Modern network environments require a fundamental shift from perimeter-centric defences to identity-centric architectures. Traditional Virtual Private Networks (VPNs) create substantial security vulnerabilities by granting broad network-level access upon initial entry while demanding complex, high-risk configurations such as open inbound ports, firewall modifications, and static public IP addresses. By replacing these legacy configurations with a Zero Trust Network Access (ZTNA) model, this project establishes a strict verification framework based on user identity, device authorization, and least-privilege access. Consequently, internal network services remain completely hidden from the public internet, neutralizing common external vector attacks.

The deployment of a low-power, lightweight Raspberry Pi as an enterprise-grade connector demonstrates that robust cybersecurity controls do not require expensive hardware investments. This edge-connector approach enables organizations to establish secure outbound tunnels to Twingate’s cloud-managed architecture without modifying existing firewall rules or altering local network topologies.

CHAPTER 2
SYSTEM ANALYSIS

2.1 LITERATURE SURVEY REVIEW

Traditional remote access frameworks heavily rely on Virtual Private Networks (VPNs) that operate on perimeter-centric defense models. Syed et al. [1] provide a comprehensive survey on Zero Trust Architecture (ZTA), emphasizing that perimeter-based models grant implicit trust to internal users, inherently violating the principle of least privilege and creating significant risks of lateral movement if credentials are compromised. Furthermore, establishing traditional VPN infrastructure requires complex and high-risk configurations, including static public IP addresses, inbound port forwarding, and continuous firewall adjustments, which expand the external attack surface.

To address perimeter vulnerabilities in constrained network environments, recent research explores Software-Defined Perimeter (SDP) models and identity-based access control. N. N. Hari et al. [2] demonstrate a standards-compliant Zero Trust framework utilizing SDP to secure residential and IoT network environments. Their findings highlight how decoupling the control plane from the data plane isolates network resources, keeping internal endpoints dark to unauthorized entities on external networks. By leveraging SDP principles, devices located behind local routers can securely connect to external services without needing exposed public IP addresses or open inbound ports.

Building on practical implementations of Zero Trust frameworks, E. Tuyishime et al. [3] evaluate the application of the Twingate platform for online laboratory access control. Their study demonstrates that Twingate provides a reliable, cloud-managed Zero Trust Network Access (ZTNA) implementation that replaces traditional VPN infrastructure with identity-verified, per-resource access policies. The work establishes that deploying Twingate simplifies access management for administrators while drastically reducing the attack surface by ensuring remote

users can only communicate with authorized applications rather than the entire subnet.

Despite the foundational insights provided by Syed et al. [1], Hari et al. [2], and Tuyishime et al. [3], a distinct gap exists in combining lightweight hardware connectors with automated resource management within an enterprise-grade ZTNA framework. While existing literature explores Zero Trust theoretically [1], applies SDP to IoT networks [2], or uses Twingate in institutional settings [3], there is limited focus on deploying a low-cost Raspberry Pi as a dedicated connector alongside API-driven dynamic resource discovery. This project addresses that gap by integrating an affordable Raspberry Pi Twingate connector with Python API automation, delivering a scalable, secure, and cost-effective remote access solution.

2.2 EXISTING SYSTEMS

2.2.1 TRADITIONAL SSL/IPSEC VPN SOLUTIONS

Traditional Virtual Private Networks (VPNs) serve as the primary legacy approach for remote network access, creating encrypted tunnels between remote devices and an internal network gateway. Once authenticated, users are granted broad network-level exposure to entire subnets, inherently violating the principle of least-privilege access and enabling lateral movement across internal resources if credentials become compromised. Furthermore, traditional VPN infrastructure requires complex and high-risk network setups, including open inbound ports, port forwarding rules, and dedicated public IP addresses, which directly expose private network services to public internet scanning.

2.2.2 PROPRIETARY ENTERPRISE ZTNA APPLIANCES

Commercial enterprise-grade Zero Trust Network Access (ZTNA) platforms utilize dedicated physical security appliances or proprietary hardware gateways to enforce identity-centric access controls. While these platforms enforce strict device verification and granular permissions, they require heavy capital expenditure and complex deployment overhead, making them impractical for lightweight, budget-

constrained edge deployments. Additionally, these traditional enterprise platforms lack native integration with affordable single-board computers like the Raspberry Pi, and their resource allocation relies on manual web console configurations rather than flexible, scriptable API automation.

2.2.3 INTERMEDIATE BASTION HOSTS AND JUMP SERVERS

Bastion hosts act as centralized intermediate proxy servers deployed within a network demilitarized zone (DMZ), requiring remote administrators to log in before jumping to internal applications via SSH or standard remote desktop protocols. Although jump servers prevent direct external visibility of internal endpoints, they introduce single points of failure and significant administrative friction. Maintaining user credentials, firewall rule tables, and host access policies requires continuous manual intervention, and the architecture lacks dynamic resource auto-discovery, limiting its ability to scale efficiently as new network services are added.

2.3 PROPOSED SYSTEM

The proposed system introduces a secure, low-cost, and efficient remote network access solution by deploying a Raspberry Pi as an on-premises connector for the Twingate Zero Trust Network Access (ZTNA) platform. Designed to eliminate the structural security risks and administrative complexities of traditional VPNs, the architecture establishes an outbound-only connection from the private network to Twingate's cloud-managed control plane. Remote users connect using the Twingate Client, where user identity and device posture are validated prior to granting access strictly to authorized resources under a least-privilege security model.

Key Attributes:

- **Zero Trust Security Model:** Implements continuous identity verification and least-privilege access control, ensuring users can only access specific, explicitly authorized resources rather than entire subnets.
- **Cost-Effective Edge Connector:** Uses a lightweight, low-power Raspberry Pi single-board computer as the physical network connector, eliminating the need for expensive enterprise hardware appliances.

- **No Inbound Exposure:** Operates via outbound-only encrypted connections, removing requirements for public IP addresses, port forwarding, or open inbound firewall ports.
- **Automated Resource Management**: Leverages Python scripts integrated with the Twingate API to automatically discover active network resources and dynamically register them without manual console updates.
- **Identity and Device Verification:** Enforces multi-factor user authentication and device authorization checks before granting access to internal resources.
- **DNS Aliasing:** Simplifies resource connectivity by allowing users to access private servers and applications via custom hostnames rather than raw IP addresses.
- **Centralized Auditing and Logging:** Maintains comprehensive records of connection events and access activities to facilitate continuous network monitoring, security assessments, and compliance auditing.

2.4 SYSTEM ARCHITECTURE OVERVIEW

The system architecture adopts a decoupled, identity-centric Zero Trust Network Access (ZTNA) framework. Rather than exposing internal services to public network traffic or relying on open inbound firewall ports, the architecture strictly separates the Control Plane (policy decisions and user authentication) from the Data Plane (encrypted traffic routing):

1. TWINGATE CONTROLLER (CLOUD CONTROL PLANE)

- Role: Centralized management, policy decision, and authentication engine, energy consumption, and meter readings) from files or the database.
- Functionality: Integrates with external Identity Providers (IdPs), evaluates access policies, generates signed Access Control Lists (ACLs) for Clients and Connectors.

2. PREPROCESSING AND FEATURE ENGINEERING

- Role: Lightweight, low-power hardware connector deployed inside the private local network.

- Functionality: Runs the Twingate Connector daemon, maintains outbound-only connections to the relay network.

3. Twingate Client (Remote User Endpoint)

- Role: Software agent running on remote user devices.
- Functionality: Intercepts traffic destined for private network resources defined in its ACL whitelist, verifies local device authorization.

4. Twingate Relay Infrastructure

- Role: NAT traversal and connection broker service.
- Functionality: Acts as a registration hub for Connectors and Clients to facilitate peer-to-peer or relayed certificate-pinned TLS tunnels across NAT boundaries without requiring public static IP addresses or open inbound ports.

5. Python API Automation Module

- Role: Programmatic resource management and dynamic network discovery script.
- Functionality: Leverages RESTful API calls to continuously scan the local subnet, auto-discover active network applications/services.

6. Internal Private Resources & Services

- Role: Protected application servers, internal web tools, databases, and administrative consoles.
- Functionality: Reside within the private local network behind the Raspberry Pi connector. Services remain completely hidden from public IP scanning.

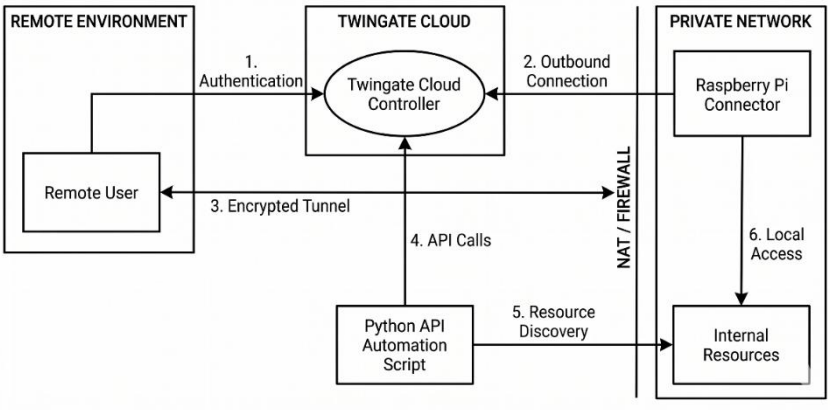


Figure 1 System Architecture

2.5 MODULE DESCRIPTION

2.5.1 Hardware Connector & Network Integration Module

This module serves as the primary physical bridge for the secure remote access architecture by deploying a Raspberry Pi as an on-premises connector. Acting as a lightweight, low-cost hardware gateway, it securely links private local network resources and internal services directly to the Twingate platform. By establishing secure outbound-only connections, this component completely eliminates the need for traditional remote access overhead—such as maintaining public IP addresses, opening incoming router ports, modifying firewall configurations, or setting up complex network routing.

2.5.2 Twingate Zero-Trust Access & Identity Control Module

This module enforces the core security framework of the project by replacing legacy VPN models with a Zero Trust Network Access (ZTNA) architecture. Operating on cloud-managed infrastructure, it centers access around explicit identity verification and strict least-privilege principles. The module manages user authentication, device status verification, dynamic access policies, and comprehensive activity logging. This ensures that authorized remote users are strictly confined to accessing only the specific internal services for which they have explicit permission, preventing lateral movement across the network.

2.5.3 Automated Resource Discovery & Registration Module

Designed to minimize manual configuration errors and optimize system scalability, this module consists of a custom Python automation engine integrated directly with the Twingate REST API. The script automatically scans the local network environment to discover active services and available network endpoints. Upon discovery, it dynamically registers these resources within the Twingate management platform, streamlining deployment and allowing administrators to

quickly onboard new internal applications or services with minimal manual intervention

2.5.4 Admin & Security Services Management Module

This module provides advanced network management features to support continuous system maintenance, remote administration, and security evaluations. It incorporates capabilities such as DNS aliasing for seamless resource addressing, service accounts for automated background workflows, and temporary, time-bound access permissions for restricted administrative tasks. Additionally, these administrative tools allow engineers to perform remote network monitoring, operational maintenance, and authorized security assessments without compromising the overall zero-trust security posture.

2.6 SYSTEM ADVANTAGES

6. Cost-Effective & Lightweight Hardware: Utilizing a Raspberry Pi as an on-premises connector creates an affordable, lightweight network bridge without requiring expensive specialized hardware.
7. Simplified Infrastructure Setup: Eliminates the need for public IP addresses, port forwarding, firewall modifications, or complex network configurations typically required by traditional VPNs.
8. Enhanced Zero-Trust Security: Operates on strict identity verification, least-privilege access, granular access policies, and device verification, preventing unauthorized access and hiding internal services from the public internet.
9. Automated & Scalable Management: Integrates Python automation with the Twingate API to automatically discover and dynamically register network resources, drastically reducing manual setup effort as the network grows.

CHAPTER 3
SYSTEM REQUIREMENTS

3.1 INTRODUCTION

The This chapter details the comprehensive hardware and software specifications essential for implementing the Secure Remote Network Access Using Twingate and Raspberry Pi architecture. Traditional remote access solutions typically rely on Virtual Private Network (VPN) tunnels that require static public IP addresses, inbound port forwarding, and manual firewall reconfigurations—practices that increase operational complexity and expand the internal network's attack surface. The proposed system mitigates these vulnerabilities by deploying a Zero Trust Network Access (ZTNA) model anchored on strict identity verification and least privilege control policies.

3.2 HARDWARE REQUIREMENTS

The following configuration is recommended for development, training, and demonstration:

Connector Node Hardware: Raspberry Pi (Model 3B+, 4, or 5 series) functions as the dedicated on-premises connector board.

Central Processing Unit (CPU): ARMv8/ARMv9 64-bit Quad-Core processor on the Raspberry Pi node for lightweight cryptography, along with an Intel Core i5 / AMD Ryzen 5 or higher processor on the administrator workstation.

Random Access Memory (RAM): 2 GB or more on the Raspberry Pi for steady network throughput and OS performance, paired with 8 GB or higher on the management machine.

Primary Storage: Class 10 / UHS-I MicroSD Card with a minimum of 16 GB capacity to store the OS, Twingate binaries, local logging, and execution scripts.

Network Interfaces: Integrated Gigabit Ethernet (RJ-45) or Dual-Band Wi-Fi (802.11ac) to establish local LAN connectivity and direct outbound cloud tunnels

Component	Specification	Description
Connector Device	Raspberry Pi (3B+, 4, or 5)	Acts as the lightweight, on-premises connector node hosting the Twingate connector agent.
Processor (Host Machine)	Intel Core i5 / AMD Ryzen 5 or higher	Required for developing API scripts, running administrative tools, and remote client management.
System RAM	2 GB+ (Raspberry Pi) / 8 GB+ (Admin Host)	Ensures smooth execution of Raspberry Pi OS, Twingate services, and administrative tasks.
Storage	16 GB MicroSD Card (Class 10 / UHS-I)	Stores the Raspberry Pi OS, Twingate connector binaries, local logs, and automated Python scripts.

Table 1. Hardware Requirements

3.3 SOFTWARE REQUIREMENTS

The software environment supports model development, data storage, explanation, and the web interface. The following configurations are recommended:

- Base Operating System: 64-bit Raspberry Pi OS (Debian-based, Lite or Desktop edition) serving as the host environment on the Raspberry Pi.
- Zero Trust Agent Infrastructure: Twingate Linux Connector Service deployed via authentication tokens to form outbound zero-trust tunnels without exposing public ports.
- Scripting & Runtime Environment: Python 3.8+ installed on the system to execute automated background workflows.
- REST API Libraries: Python requests library used to interface programmatically with the Twingate REST API for automated IP discovery and network resource registration.
- Cross-Platform Access Clients: Twingate Client Applications running on administrator and end-user platforms (Windows, macOS, Linux, Android, iOS).
- Administration & Diagnostic Utilities: Native Linux networking tools including OpenSSH, Nmap, and Terminal utilities for remote node management, network mapping, and security auditing

Component	Software / Tool	Specification / Version	Description
Operating System	Raspberry Pi OS	Debian-based (64-bit with Desktop or Lite)	Host operating system running on the Raspberry Pi connector node.

Zero-Trust Access Platform	Twingate Connector	Latest Stable Linux Connector Agent	Lightweight connector service deployed via terminal tokens to establish outbound Zero-Trust tunnels.
Automation & API Engine	Python	Version 3.8 or higher	Executes automation scripts to interface with the Twingate REST API for automated IP discovery and dynamic resource provisioning.
API Client Libraries	Python requests / http.client	Latest Stable Release	Facilitates HTTP authentication, token verification, and RESTful endpoint management with Twingate's cloud API.
Remote Client Software	Twingate Client Application	Windows, macOS, Linux, iOS, Android	End-user software enabling authenticated devices to securely connect to published internal resources based on granular access policies.

Administrative Tools	OpenSSH / Nmap / Terminal	Standard Linux Tooling	Used for initial headless configuration, internal network scanning, resource verification, and security testing.

Table 2.Software Requirements

3.4 FUNCTIONAL REQUIREMENTS

The functional requirements define the specific features, capabilities, and operational workflows that the secure remote access system must perform.

- **Zero-Trust Connection Management:** The system must establish secure, encrypted outbound tunnels from the Raspberry Pi connector to the Twingate cloud controller without opening inbound firewall ports or requiring a public IP address.
- **Identity-Based Authentication**: The platform must verify user identities and apply least-privilege access controls before granting access to internal network resources.
- **Automated Network Resource Discovery:** The Python automation engine must query local network details via the Twingate REST API to automatically detect the connector's IP address and programmatically register available services.
- **Granular Access Policy Enforcement:** Administrators must be able to define and enforce fine-grained access rules based on specific protocols, target ports, session expiration timers, and user/group memberships.

- **Device Compliance & Verification:** The system must evaluate connecting client devices against security requirements (such as firewall status and disk encryption) before granting resource access.
- **Multi-Platform Client Connectivity**: Cross-platform Twingate client applications (Windows, macOS, Linux, iOS, Android) must allow authenticated users to view and connect only to their authorized resources.
- **Remote Network Diagnostic Support:** The connector node must allow administrators to execute internal diagnostic tools (such as SSH or network scanning) remotely to manage and support target network infrastructure.

3.5 NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements specify the system's quality attributes such as reliability, scalability, efficiency, and usability. They determine how effectively the functional objectives are achieved.

- **Performance:** Ensures fast handshake setup times and low latency data routing through outbound encrypted tunnels without introducing network bottlenecks.
- **Scalability:** Easily expands across multi-site networks by deploying additional lightweight Raspberry Pi connectors and dynamic API registrations.
- **Usability:** Delivers an intuitive cross-platform client experience for end-users and a simplified "drop and forget" installation process for administrators.
- **Reliability:** Guarantees stable remote connectivity by leveraging cloud orchestration and failover capabilities across multiple connector instances. Security: Enforces strict Zero Trust Network Access (ZTNA) policies,

keeping internal network infrastructure completely hidden from public port scanners.

- **Auditability:** Provides detailed logging of session histories, active users, and resource interactions for security compliance and incident reviews.
- **Maintainability:** Relies on standardized Python scripts and cloud-managed configurations to reduce manual overhead and simplify troubleshooting

CHAPTER 4
SYSTEM DESIGN AND IMPLEMENTATION

4.1 OVERVIEW

This chapter details the structural design and practical execution of the Remote Network Access Using Twingate and Raspberry Pi project. The proposed architecture eliminates traditional virtual private network (VPN) vulnerabilities—such as open inbound ports, static public IP requirements, and perimeter-only access controls—by establishing an identity-based Zero Trust Network Access (ZTNA) model. The implementation spans physical connector setup on single-board hardware, cloud-orchestrated identity and policy configuration, Python API integration for dynamic resource discovery, and client-side access control.

4.2 SYSTEM ARCHITECTURE

The system architecture consists of three distinct layers working synchronously to facilitate secure, peer-to-peer remote access without exposing the target local network to the public internet.

Architecture Components:

The architecture of the system is designed with a modular structure to ensure efficiency, scalability, and easy integration of various functionalities.

1. **Control Plane (Twingate Cloud Controller):**

   Handles user authentication, identity verification, device compliance checks, and access rule distribution. It functions strictly as an orchestration engine and does not inspect or route internal user data traffic.

2. **Connector Plane (On-Premises Raspberry Pi):**

   A low-power Raspberry Pi running a headless Linux connector agent. It establishes a continuous, secure outbound-only connection to the Twingate

Cloud, allowing remote client traffic to enter the target LAN without modifying router firewall settings or configuring port forwarding.

3. **Data Plane & Automated Discovery Engine:**

   When an authenticated user requests access to an internal network asset, the Twingate Client establishes a direct encrypted peer-to-peer data tunnel to the Raspberry Pi connector. Additionally, an integrated Python script uses Twingate REST APIs to scan local subnets, retrieve device IP addresses, and programmatically register resources within the controller

4.3 WORKFLOW DESCRIPTION

The operational workflow of the secure remote access system spans six sequential phases, executing a end-to-end process from hardware deployment to authenticated data tunnelling.

Step-by-Step Workflow:

4. **Connector Provisioning & Network Attachment:**The Raspberry Pi is flashed with Raspberry Pi OS, pre-configured with hostname, credentials, and network interfaces (Ethernet or Wi-Fi). Upon powering up inside the target remote network, the board establishes outbound internet connectivity.
5. **Cloud Controller Orchestration & Token Deployment**:An administrator creates a remote network entry within the Twingate Cloud Controller console. A unique, cryptographic deployment token is generated and executed on the Raspberry Pi terminal, initializing the Twingate Connector service. The connector then establishes a secure, outbound-only registration tunnel to the Twingate cloud without requiring open inbound ports or static public IPs.
6. **Automated Resource Discovery & API Registration:**To eliminate manual configuration overhead, a custom Python automation script authenticates with Twingate's REST API. The script queries the connector’s local network environment, detects active LAN devices and IP addresses,

and programmatically registers these entities as accessible network resources in the Twingate portal.

7. **Access Control & Security Policy Enforcement:**The administrator assigns granular access rules to the registered resources based on the principle of least privilege. Rules specify user/group authorizations, allowed protocol ports (e.g., SSH, HTTP/HTTPS), session duration timers, and client device compliance standards (such as active firewalls and disk encryption).

8. **Client Authentication & Mutual Handshake:**A remote user launches the Twingate Client application on their device (Windows, macOS, Linux, iOS, or Android) and authenticates through the identity provider. Upon successful verification, the cloud controller evaluates the user's permissions and issues authorization tokens exclusively for the resources the user is permitted to access.

9. **Encrypted Data Tunneling & Audit Logging:**When the user attempts to connect to an internal resource, the Twingate Client and the Raspberry Pi connector establish a direct, end-to-end encrypted peer-to-peer data tunnel. The connector proxies the authorized requests to the local destination while all session activity, connection metadata, and access timestamps are logged for security auditing.

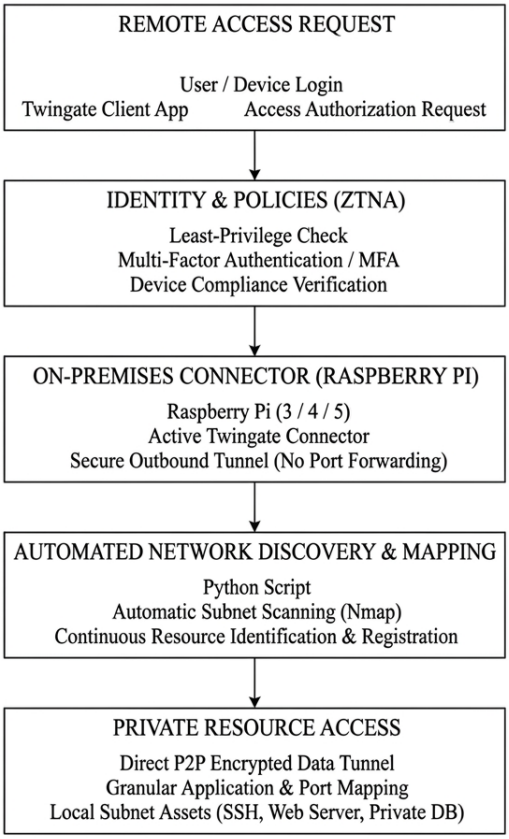


Figure 2 Overall System Workflow

4.4 MODULES AND THEIR INTERACTIONS

The major modules of the system and their interactions are described below. Each module performs a distinct role but is interconnected to achieve the common goal of reliable, explainable theft detection.

4.4.1 **Hardware & OS Setup Module**

- Involves flashing Raspberry Pi OS onto an SD card and configuring initial settings like hostname, credentials, and network interface connections.
- Deploys the Raspberry Pi as a lightweight internal access point inside the target network.

4.4.2 **Twingate & ZTNA Integration Module**

- Establishes the remote network in the Twingate console and generates installation tokens.
- Connects the Raspberry Pi using secure outbound tunnels without changing firewall or NAT rules.

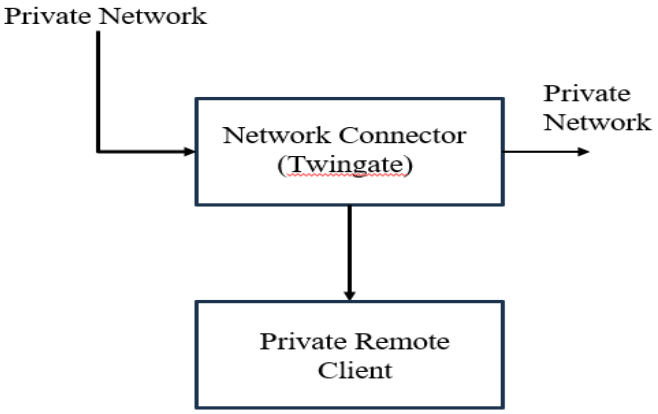


Figure 3.Twingate Integration Module

4.4.3 **Network Discovery & Automation Module**

- Runs a custom Python script to autodiscover the Pi's local and public IP addresses.
- Automatically registers target internal devices (servers, routers) to Twingate without manual entry.

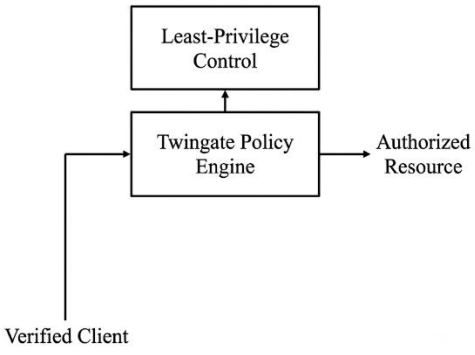


Figure 4.Network Policy Module

4.4.4 **Access Control & Policy Module**

- Restricts user access based on explicit group permissions, protocols, ports, and session times.
- Enforces device compliance (e.g., encryption) while keeping detailed activity logs.

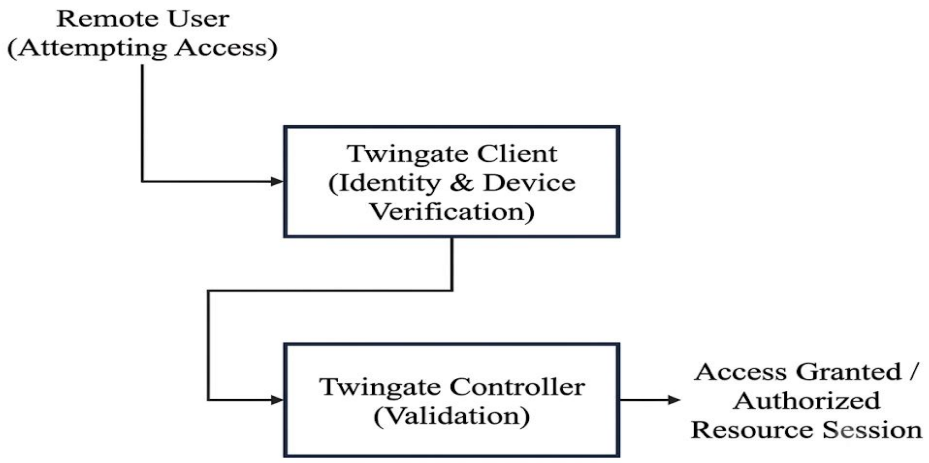


Figure 5.Autharization Module

4.4.5 Client & Monitoring Integration Module

- Connects desktop and mobile users securely using native Twingate clients and DNS aliases.
- Configures headless accounts for third-party monitoring and supports multiple failover connectors.

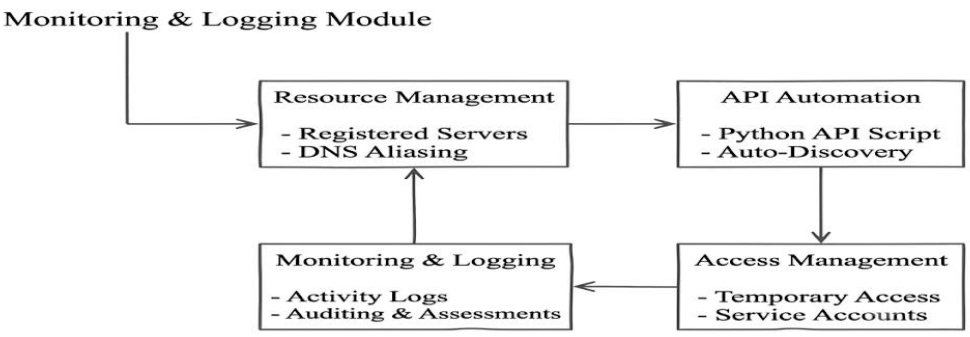


Figure 6.Monitoring and Logging Module

4.5 IMPLEMENTATION DETAILS

Deploy a Raspberry Pi as a headless zero-trust connector running Twingate, using Python API scripts to auto-register internal network resources for secure remote client access.

Hardware & OS Setup:

- Device: Raspberry Pi (3/4/5) running Raspberry Pi OS flashed via Raspberry Pi Imager.
- Network Integration: Connects via Ethernet/Wi-Fi to the target network as a plug-and-play headless node.

Twingate Setup:

- Cloud Console: Network location created in the Twingate cloud portal to generate deployment tokens.
- Connector Token: Connector agent installed on the Pi via terminal command to form secure outbound-only tunnels.

API & Automation:

- Python Script: Interacts with Twingate's API to auto-detect the Pi's internal and external IPs.
- Auto-Discovery: Automatically maps and registers local devices (routers, servers) into Twingate.

Security Policies:

- Access Rules: Enforces least-privilege access restricted by user group, port, protocol, and session time.
- Auditing: Monitors user activity and checks client device security compliance (encryption, firewall).

Client Connectivity:

- End-User Apps: Twingate client installed on desktops and mobile devices (Windows, Mac, Linux, iOS, Android).

- Service Accounts: Dedicated non-human credentials created for automated network monitoring tools.

4.6 CHALLENGES AND MITIGATION STRATEGIES

Challenge	Description	Mitigation Strategy
Connector Disconnection	Physical power loss, SD card corruption, or unexpected reboot of the Raspberry Pi offlines remote access.	Implement watchdog timers, auto-reboot scripts, and deploy redundant multi-connector Pi setups.
Dynamic Public IP Changes	Target network ISP changes public IP address, disrupting traditional connection methods.	Utilize Twingate's cloud architecture and Python API script to auto-discover and update IP changes dynamically.
Unauthorized Access & Identity Abuse	Compromised user credentials could grant unauthorized access to internal resources.	Enforce Zero Trust Architecture, MFA, strict device security compliance, and session expiration timers.
Over-Privileged Access	Broad access permissions could expose entire subnets if a single device is compromised.	Enforce least-privilege policies by scoping access to exact IP addresses, specific hostnames, and required ports.

Table 3. Challenges and Mitigation Strategies

CHAPTER 5
ETHICAL CONSIDERATIONS AND LIMITATIONS

5.1 INTRODUCTION

The deployment of remote access solutions, particularly low-cost hardware devices like the Raspberry Pi operating within local environments, presents critical ethical and security considerations that must be systematically addressed. While combining Raspberry Pi hardware with Twingate's Zero Trust Network Access (ZTNA) platform significantly improves network accessibility, automated resource discovery, and remote administration, it also introduces responsibilities regarding unauthorized deployment, network privacy, and privilege management. Deploying pre-configured hardware connectors without explicit organizational authorization or proper governance can create unintended security risks or covert access points.

5.2 ETHICAL CONSIDERATIONS

5.2.1 Data Privacy And Security

Deploying a Zero Trust Network Access (ZTNA) connector inside a network requires strict data privacy governance to prevent unauthorized data exposure. Because the Twingate connector running on the Raspberry Pi acts as an encrypted gateway to internal resources, it routes metadata, IP addresses, and user activity through cloud infrastructure. To uphold privacy standards, end-to-end encryption must be maintained across all tunnels so that sensitive network traffic and user payloads remain inaccessible to third-party relays.

5.2.2 Fairness and Bias

Ethical network administration demands that access control policies are applied fairly and objectively across all authorized personnel. Utilizing Twingate's central administrative controls allows organization-wide role-based access rules to be enforced uniformly, eliminating arbitrary or discriminatory access restrictions. By

relying on standardized identity authentication and predefined group policies rather than discretionary manual configurations, the system ensures that every user is granted access solely on the basis of operational necessity and verified role permissions.

5.2.3 Transparency and Explainability

Maintaining transparency within the network access architecture is essential for both administrative clarity and operational trust. The integration of Python scripts utilizing Twingate's API automates network resource mapping and address registration, making it imperative that automated decisions remain fully explainable to system administrators. Clear documentation of script behaviors, open API configurations, and accessible resource lists ensure that network topology changes are easily interpretable.

5.2.4 Accountability and Human Oversight

While API scripts and headless Raspberry Pi devices automate routine discovery and connection tasks, complete accountability must remain with human administrators. Automated network provisioning can lead to unintended access exposure if left entirely unsupervised. To mitigate this, human oversight is integrated at critical checkpoints, requiring administrative validation before newly discovered resources or administrative permissions are permanently committed.

5.2.5 Environmental and Societal Impact

From an environmental perspective, utilizing low-power hardware like the Raspberry Pi significantly reduces energy consumption compared to deploying standard, full-sized server hardware for remote access routing. This energy-efficient profile minimizes the overall carbon footprint of continuous 24/7 network monitoring setups. Societally, providing a secure, affordable zero-trust solution democratizes access to enterprise-grade security for small businesses, community organizations, and non-profits that lack dedicated IT budgets, enabling secure remote collaboration without compromising network integrity.

5.3 LIMITATIONS OF THE SYSTEM

5.3.1 Data Dependency

The performance and accuracy of automated resource provisioning within this architecture rely heavily on consistent network metadata. The custom Python script relies on predictable local IP schemes, accurate DNS entries, and clear API responses from the Twingate controller to dynamic resource registration.

5.3.2 Class Imbalance

Within security logging and continuous access evaluation, standard operational activity drastically outnumbers potential security violations or unauthorized access attempts. Because benign connection logs dwarf malicious or anomalous events, automated alerting systems and policy engines risk overlooking subtle unauthorized access patterns or suffering from alert fatigue.

5.3.3 Detection of Sophisticated Attacks

While Zero Trust Network Access (ZTNA) eliminates traditional perimeter vulnerabilities and prevents lateral network movement, it is not inherently designed to detect advanced, multi-stage application-layer attacks. If an attacker steals valid user credentials, passes multi-factor authentication, and satisfies client device security requirements, the ZTNA platform will grant access to authorized resources.

5.3.4 Concept Drift

Over time, network environments, user roles, device configurations, and active service endpoints undergo continuous changes—a process known as concept drift. Static Twingate resource mappings and Python API discovery rules can gradually become out-of-date as network architecture evolves.

5.3.5 Software-Only Scope

The security guarantees of this solution operate entirely at the software layer (OS, agent, and cloud relay layers), leaving physical hardware vulnerabilities unaddressed. Because the Raspberry Pi sits on-site as an edge connector, it remains

vulnerable to physical tampering, SD card extraction, power disruptions, or hardware-level tapping. A software-only architecture cannot prevent physical theft or direct hardware manipulation of the connector node in unmonitored environments.

5.3.6 Transferability across Regions

Deploying the solution across different geographic regions presents operational constraints related to cloud latency and regional regulatory compliance.

5.4 MITIGATION STRATEGIES

Limitation	Mitigation Strategy
Data dependency	Implement fallback local IP scanning mechanisms, robust error-handling routines in the API script, and fallback manual entry options for unmanaged subnets.
Class imbalance	Apply log filtering rules, anomaly-based threshold alerting, and weighted severity scoring to prioritize critical security violations over routine connections.
Sophisticated attacks	Integrate endpoint detection and response (EDR) software on connected client devices and export Twingate audit logs to a central SIEM for deep packet and behavior analysis.
Concept drift	Schedule periodic automated network re-scans via Python scripts and enforce mandatory quarterly reviews of active permissions, resources, and service accounts.
Software-only scope	Enforce physical security controls such as tamper-evident enclosures, disabled physical USB ports, encrypted storage, and network port security (802.1X).

Table 4 Mitigation Strategies

5.5 SUMMARY

This chapter evaluated the ethical obligations and systemic constraints inherent in deploying a Zero Trust Network Access (ZTNA) system using a Raspberry Pi hardware connector and automated Python API scripts. The ethical framework emphasized maintaining strict data privacy through end-to-end encryption, ensuring unbiased role-based access control, keeping automated discovery processes transparent, and preserving human oversight over all administrative actions. Additionally, the chapter analyzed key system limitations—including data dependency on local network configurations, log class imbalance, vulnerabilities to sophisticated credential abuses, concept drift over time, software-only scope risks, and regional latency variations. To address these challenges, comprehensive mitigation strategies were structured to establish physical security measures, automated re-scans, SIEM log integrations, and regional relay optimizations, ensuring the overall deployment remains secure, transparent, and resilient.

CHAPTER 6
CONCLUSION AND FUTURE SCOPE

6.1 CONCLUSION

The project successfully demonstrates a low-cost, efficient, and highly secure remote network access solution by integrating a Raspberry Pi connector node with Twingate's Zero Trust Network Access (ZTNA) platform. Traditional remote management solutions reliance on legacy VPNs often introduces operational friction, complex port forwarding, and broad network exposure. In contrast, this system establishes a drop-in zero-trust architecture that operates entirely through secure, outbound-only encrypted tunnels. By removing the need for public IP addresses or static firewall adjustments, the deployment significantly shrinks the external attack surface while streamlining administration across heterogeneous cross-platform environments.

Furthermore, the incorporation of Python API automation dramatically improves system scalability and eliminates manual configuration errors. The custom scripts programmatically discover internal network interfaces and register target hosts directly into the Twingate cloud controller. Coupled with fine-grained access policies, least-privilege permission controls, DNS aliasing, and non-human service accounts for headless monitoring tools, the framework delivers enterprise-grade security oversight at a fraction of standard infrastructure costs. Ultimately, this project proves that combining lightweight single-board computers with cloud-managed ZTNA platforms offers a practical, scalable, and auditable pathway for modern remote IT support, security assessments, and organizational network management.

6.2 FUTURE SCOPE

Future enhancements for this project include integrating automated intrusion detection capabilities directly onto the Raspberry Pi edge device using lightweight security monitoring tools like Suricata or Snort. The network discovery scripts can

be expanded to leverage machine learning models to dynamically categorize discovered network hosts and automatically suggest appropriate access policies and port restrictions.

6.2.1 Automated Multi-Connector Failover

Developing a dynamic peer-to-peer heartbeat protocol among multiple Raspberry Pi nodes to ensure seamless high availability and automatic route failover during hardware or power outages.

6.2.2 Real-Time Network Telemetry & Auto-Mapping

Enhancing the Python API automation to continuously listen for local network changes (via passive ARP/DHCP monitoring) and instantly sync newly connected hosts to Twingate without manual script execution.

6.2.3 SIEM & Endpoint Detection Integration

Establishing automated log forwarding pipelines to feed Twingate access events directly into centralized security dashboards (e.g., Splunk, Elastic, or Wazuh) alongside endpoint security checks.

6.2.4 AI-Driven Threat & Anomaly Detection

Integrating lightweight local machine learning models on the Raspberry Pi to analyze local network telemetry and detect unusual traffic patterns or unauthorized lateral movement attempts in real time.

6.2.5 Containerized Hardware Expansion

Packaging the connector environment into lightweight Docker and OpenWrt container images to allow effortless deployment across diverse hardware, including travel routers, edge gateways, and NAS units.

REFERENCES

1. Syed et al., “Zero Trust Architecture (ZTA): A Comprehensive Survey,” IEEE Access, 2022.
2. N. N. Hari et al., “Standards-Compliant Zero Trust Framework-Based Software-Defined Perimeter (SDP) for Securing Residential IoT Networks,” IEEE.
3. E. Tuyishime et al., “Online Laboratory Access Control With Zero Trust Approach: Twingate Use Case,” IEEE.
4. M. Zohaib et al., “Zero Trust VPN (ZT-VPN): A Systematic Literature Review and Cybersecurity Framework for Hybrid and Remote Work,” Information, MDPI / IEEE Indexed, 2024.
5. M. S. Inukonda et al., “ZETA: A Zero-Trust Security based Forensic-Ready Solution for Perimeter-less Enterprise Networks,” IEEE International Conference on Advanced Networks and Telecommunications Systems (ANTS), 2023.
6. H. Zhang et al., “Toward Zero-Trust IoT Networks via Per-Packet Authorization,” IEEE Communications Magazine, 2024.
7. M. Hussain et al., “Federated Zero Trust Architecture using Artificial Intelligence,” IEEE Wireless Communications, 2024.
8. J. Weber et al., “Lightweight WebAssembly-Based Intrusion Detection for Zero Trust Edge Networks,” IEEE Access, 2026.

APPENDIX

Appendix A: System Architecture Diagram

The system architecture diagram illustrates the end-to-end data flow and connectivity pipeline of the proposed Zero Trust Network Access (ZTNA) solution. It visually represents the sequential stages starting from local Raspberry Pi deployment within the private network, establishing outbound encrypted tunnels to the Twingate cloud controller, executing API-driven network resource discovery, applying granular least-privilege policies, and providing secure client access across remote endpoints.

Each module performs a specialized role:

- Configures the Raspberry Pi OS environment and establishes local network connectivity via Ethernet or Wi-Fi.
- wingate Connector Layer: Establishes secure outbound-only TLS tunnels to Twingate relays without requiring open firewall ports or port forwarding.
- Network Discovery & Automation Module: Runs Python scripts interacting with the Twingate API to auto-detect internal IP ranges and register network devices dynamically.

Appendix B: Configuration & Metadata Field Description

The framework operates on structured network parameters, identity attributes, and API resource definitions. The principal fields utilized across the Twingate platform and automation scripts are listed below:

- **Connector ID & Token:** Unique identifier and deployment key issued by the Twingate controller to authenticate the Raspberry Pi hardware node.
- **Private & Public IP Address:** Local subnet address and external public address auto-discovered by the Python script to map the connector's location.
- **Resource Hostname**: Internal domain name or custom DNS alias mapped to a specific target network device (e.g., router, printer, server).

Appendix C: Security & Operational Metrics

System evaluation relies on standardized security criteria, network performance indicators, and access control audit metrics:

- **Attack Surface Reduction:** Extent to which open inbound firewall ports and public IP exposure are eliminated.
- **Authentication Latency:** Time required to verify user identity, check device security posture, and establish an encrypted session.
- **Least-Privilege Coverage:** Proportion of network resources restricted to specific ports and users rather than granting full subnet access.